

Методичка: запуск задачи KUKA youBot «детекция → захват → перестановка по цвету»

Документ охватывает четыре раздела:

- **A — Запуск симуляции в Gazebo**
- **B — Запуск на реальном роботе** (с пометкой, ГДЕ запускать каждую строку)
- **C — Короткие команды для теста** (схват на максимум, медленные скорости, проверка камеры/детекции)
- **D — Трешшутинг** (EtherCAT, инициализация робота и прочее)

Важнейшее, что нужно понять до запуска

Единый файл параметров

Все важные значения задачи лежат в ОДНОМ файле:

```
src/youbot_perception/config/mission_params.yaml
```

Точки A/B/C, цвет кубика, радиус/разброс/частота препятствий, seed, габарит робота, скорости, допуски, число попыток детекции — всё там. Меняете значение в этом файле → оно подхватывается всеми нодами (через namespace `/mission_config`).

Любой параметр можно переопределить «на лету» аргументом launch, не трогая файл — см. разделы A и B.

Система координат (КРИТИЧЕСКИ важно для реального робота)

- Мировой кадр задачи — `odom`. В нём заданы точки A/B/C, препятствия, позиция кубика.
- На реальном роботе кадр `odom` создаётся драйвером в тот момент, когда вы запускаете `youbot_driver`. **Начало координат `odom` = там, где стоит робот в момент старта драйвера, а ось +X `odom` смотрит туда, куда смотрит «нос» робота (сторона руки манипулятора).**
- В проекте принята геометрия: **рука манипулятора — по +X базы, камера — по -X базы** (камера и рука с противоположных сторон). Миссия сама разворачивает базу: камерой (-X) к кубику при осмотре, рукой (+X) к кубику при захвате.

Из этого следует правило установки робота (см. раздел B, шаг «Как ставить робота»).

Схват

- **Симуляция (URDF):** пределы пальца жёстко заданы в `youbot_description/.../limits.urdf.xacro` как `0.023/2 = 0.0115 м` на палец, т.е. максимум ~23 мм суммарно. В коде `GRIPPER_OPEN = 0.0115` (на сторону) — это «полностью открыт» в симуляции.
- **Реальный робот:** физический ход схвата youBot больше (полное раскрытие порядка **7 см** суммарно, т.е. ~0.035 м на палец). Команда схвату задаётся в метрах НА ПАЛЕЦ. Чтобы открыть реальный схват на максимум, публикуйте ~0.035 на каждый палец (см. раздел C). Кубик ~5 см — в симуляции схват его не обхватывает (берём за ребро), на реальном — обхватывает.

A. Запуск симуляции в Gazebo

A.0 Предусловия (один раз)

1. Рабочее окружение: Ubuntu 20.04 + ROS Noetic, собранный catkin-воркспейс с этим проектом.
2. Установлены зависимости симуляции и детекции:

```
bash sudo apt install ros-noetic-gazebo-ros ros-noetic-gazebo-ros-control \ ros-noetic-controller-manager ros-noetic-joint-trajectory-controller \ ros-noetic-effort-controllers ros-noetic-robot-state-publisher \ ros-noetic-cv-bridge python3-opencv
```
3. Плагин grasp-fix (`libgazebo_grasp_fix.so`) — нужен, чтобы кубик «прилипал» к схвату в Gazebo (`gazebo_grasp_plugin`). Если его нет — захват в симуляции будет ненадёжным.

A.1 Сборка

```
cd ~/catkin_ws
catkin build          # или catkin_make
source devel/setup.bash
```

A.2 Базовый запуск (комната 5×5 по умолчанию)

```
roslaunch youbot_perception pick_place_sim.launch
```

Что произойдёт по шагам (всё в одном launch): 1. поднимется Gazebo (пустой мир); 2. загрузится youBot с USB-камерой и плагином `planar_move`; 3. поднимутся контроллеры руки и схвата; 4. `room_generator` построит комнату, сетку, точки A/B/C и препятствия; 5. `spawn_task_objects` поставит кубик в B и три картонки (red/green/blue) в C; 6. `cube_detector` начнёт детекцию цвета по камере; 7. через ~10 с стартует `pick_place_mission` — едет к B, определяет цвет, берёт кубик, везёт к C, ставит на картонку нужного цвета, возвращается в A.

A.3 Свой размер комнаты (под реальное пространство)

Параметры комнаты можно задать прямо в команде (переопределяют YAML):

```
roslaunch youbot_perception pick_place_sim.launch \
  room_width:=3.0 room_height:=2.5 cell_size:=0.5
```

- `room_width` — размер вдоль X (м), `room_height` — вдоль Y (м).
- `cell_size` — размер ячейки сетки (м). Число ячеек = размер / `cell_size` (округляется). Для 3.0×2.5 при `cell_size=0.5` получится сетка 6×5.

A.4 Свой выбор ячеек кубика, платформ и робота

Точки задаются НОМЕРОМ ЯЧЕЙКИ в формате "`col,row`" (колонка, строка), отсчёт от 0 из левого-нижнего угла комнаты:

```
roslaunch youbot_perception pick_place_sim.launch \
  cell_A:=0,0 cell_B:=2,4 cell_C:=5,4 \
  robot_x:=0.5 robot_y:=0.5
```

- `cell_A` — парковка/home (старт и финиш робота);
- `cell_B` — кубик (pick);
- `cell_C` — склад (place, три картонки рядом);
- `robot_x`, `robot_y` — поза спавна робота в Gazebo. **Должна совпадать с центром ячейки A.** Центр ячейки = $\text{origin} + (\text{col} + 0.5) * \text{cell_size}$. Для `cell_A=0,0` и `cell_size=1.0` это (0.5, 0.5).

Если оставить `cell_A/B/C` пустыми — они выберутся автоматически (A в углу у (0,0), C в дальнем углу, B в центре).

Пример «всё своё»: комната 4×3, ячейки 0.5 м, кубик в (3,5), склад в (7,5), парковка в (0,0), робот стартует в центре A:

```
roslaunch youbot_perception pick_place_sim.launch \
  room_width:=4.0 room_height:=3.0 cell_size:=0.5 \
  cell_A:=0,0 cell_B:=3,5 cell_C:=7,5 \
  robot_x:=0.25 robot_y:=0.25
```

(центр ячейки 0,0 при `cell_size=0.5` = 0.25, 0.25)

A.5 Фиксированный цвет кубика и seed

```
roslaunch youbot_perception pick_place_sim.launch cube_color:=blue seed:=7
```

- `cube_color` ∈ {red, green, blue}; пусто => случайный (по seed);
- `seed` фиксирует расстановку препятствий и случайный цвет (воспроизводимость).

A.6 Препятствия (радиус, разброс, частота)

Эти три величины меняются в `mission_params.yaml`, группа `obstacles`: - `radius_min` / `radius_max` — радиус цилиндра (если равны — фиксированный, сейчас 0.1 м); - `spread` — разброс центра препятствия внутри ячейки (0 = строго центр, 1 = максимальный разброс к краям с учётом зазора); - `max_per_cell` — максимум цилиндров в ячейке (частота появления, 0..max).

А.7 Полезные топики для наблюдения

```
rostopic echo /cube/color           # определённый цвет
rostopic echo /cube/pose            # позиция кубика в odom
roslaunch rqt_image_view rqt_image_view /cube/debug_image # обводка кубика
rostopic echo /mission_planned_path # ПЛАНИРУЕМЫЙ путь (из RRT)
rostopic echo /mission_actual_path  # ФАКТИЧЕСКИ пройденный путь (из /odom)
```

А.8 Визуализация в RViz и запись видео (сравнение симуляции и реальности)

В проекте есть готовая конфигурация RViz, которая показывает всю сцену в кадре `odom`:

- **комната, стены, сетка, препятствия, точки A/B/C** — `/room/markers`;
- **угловые маркеры рабочей зоны** — `/field/markers`;
- **запланированный путь (зелёный)** — `/mission_planned_path`;
- **фактически пройденный путь (красный)** — `/mission_actual_path`;
- **модель робота и debug-изображение камеры** — `/cube/debug_image`.

Зелёный путь — это план RRT; красный — реальный след базы по одометрии. Их наложение и есть наглядное «план vs реальность». Тот же красный след в Gazebo и на реальном роботе позволяет сравнить симуляцию с реальностью по видео.

Способ 1 — RViz вместе с запуском симуляции (одной командой):

```
roslaunch youbot_perception pick_place_sim.launch rviz:=true
```

Способ 2 — RViz отдельным окном (к уже запущенной сцене, в т.ч. на реальном роботе):

```
roslaunch youbot_controller rviz.launch
```

Запись видео: - экран Gazebo: встроенная запись `Window → Video → Record`, либо любой экранный рекордер ОС; - сцена RViz: экранный рекордер ОС (например `recordmydesktop`, OBS, GNOME «Снимок экрана» с записью); - сравнение: запишите прогон в Gazebo (`pick_place_sim.launch rviz:=true`) и прогон на реальном роботе с тем же RViz-конфигом (`youbot_controller rviz.launch`) — красный фактический путь в обоих случаях строится из `/odom`, поэтому траектории прямо сопоставимы.

Альтернатива видео — записать бэг и проиграть позже: `bash rosbag record /odom /mission_planned_path /mission_actual_path \ /room/markers /cube/debug_image /tf -O run.bag`

проигрывание:

```
rosbag play run.bag # параллельно открыть roslaunch youbot_controller rviz.launch ``
```

В. Запуск на реальном роботе

В этом разделе у каждой строки стоит пометка, ГДЕ её запускать:

- **[НОУТБУК]** — на вашем ноутбуке (рабочая станция оператора);
- **[YUBOT]** — на бортовом компьютере KUKA youBot (там, где стоит `youbot_driver` и куда подключены моторы по EtherCAT).

Бортовой компьютер youBot и ноутбук должны быть в одной сети ROS. Обычно бортовой ПК — это `ROS_MASTER`. Драйвер EtherCAT поднимается ТОЛЬКО на бортовом ПК, потому что только к нему физически подключены контроллеры моторов.

В.0 Как ставить робота (СИСТЕМЫ КООРДИНАТ)

Это самый важный шаг для реального робота — от него зависит, совпадут ли координаты задачи с реальностью.

1. **Выберите физическую точку парковки А в комнате** — это место, где робот будет стоять в момент старта драйвера. Лучше всего — угол комнаты, соответствующий ячейке А (по умолчанию (0,0), т.е. левый-нижний угол).

2. **Поставьте робот точно в эту точку** так, чтобы: - «нос» робота (сторона руки манипулятора, +X) смотрел вдоль оси +X вашей комнаты (в сторону возрастания координаты X, т.е. к точке C/дальному углу при стандартной раскладке); - камера (-X, противоположная руке сторона) смотрела назад, от комнаты.
3. **Сориентируйте комнату в коде под эту установку.** Начало `odom` появится ровно там, где стоит робот, а +X `odom` — туда, куда смотрит рука. Значит: - origin комнаты (`room/origin_x` , `origin_y`) = (0,0) совпадёт с роботом; - ось X комнаты пойдёт «вперёд» от робота, ось Y — влево.
4. Если физически удобнее ставить робота иначе — поверните раскладку ячеек (cell_A/B/C) и/или origin в конфиге так, чтобы они отражали реальное расположение кубика и склада ОТНОСИТЕЛЬНО стартовой позы робота.

Практическое правило: **сначала ставите робот и запоминаете, где у него «перёд» (рука), потом описываете комнату в этих же осях.** Кубик, который физически находится «впереди-слева» от стартовой позы робота, должен быть в ячейке с большим Y и небольшим X.

В.1 Подготовка железа (один раз перед сессией)

1. **[YOUBOT]** Убедитесь, что база youBot включена (свинцовый аккумулятор), манипулятор запитан, EtherCAT-кабель цел.
2. **[НОУТБУК]** Проверьте связь с роботом: `bash ping 10.0.0.1 # IP бортового ПК youBot (пример)`
3. **[YOUBOT]** Один раз решите проблему прав EtherCAT (детали — раздел D): `bash # узнать реальное имя интерфейса к моторам (НЕ обязательно eth0!) ip link # прописать его в конфиг (пример enp2s0): nano $(rospack find youbot_driver)/config/youbot-ethercat.cfg # выдать права бинарию драйвера (повторять после каждой пересборки!) sudo setcap cap_net_raw,cap_net_admin+eip \ $(catkin_find youbot_driver_ros_interface youbot_driver_ros_interface | head -1)`

В.2 Сеть ROS (каждый терминал)

Договоритесь, кто `ROS_MASTER`. Пусть мастер — бортовой ПК youBot.

- **[YOUBOT]** (терминал с мастером): `bash export ROS_MASTER_URI=http://10.0.0.1:11311 export ROS_IP=10.0.0.1 roscore # либо master поднимется из roslaunch ниже`
- **[НОУТБУК]** (каждый терминал на ноутбуке): `bash export ROS_MASTER_URI=http://10.0.0.1:11311 export ROS_IP=<IP_вашего_ноутбука_в_этой_сети>`

В.3 Запуск драйвера робота

- **[YOUBOT]** Поставить робот в точку A (см. В.0), затем: `bash source ~/catkin_ws/devel/setup.bash roslaunch youbot_driver_ros_interface youbot_driver.launch` Этот драйвер поднимает базу (+ `/odom` , `/cmd_vel`), руку и схват. **Именно в этот момент фиксируется начало координат `odom`** — робот должен уже стоять правильно.

Если видите `No socket connection on ...` или `No EtherCAT slaves found` — раздел D.

В.4 Камера и её положение в кадре координат

- **[НОУТБУК]** Запустите USB-камеру и задайте TF её крепления. Проще всего — целиком через основной launch (он сам поднимет `usb_cam` и static TF): `bash source ~/catkin_ws/devel/setup.bash roslaunch youbot_perception pick_place_real.launch video_device:=/dev/video0` Этот launch на ноутбуке запускает: `usb_cam` , статический TF `base_link` → `usb_cam_optical_frame` , `cube_detector` и `pick_place_mission` .

ВАЖНО про TF камеры: в launch задан примерный трансформ `args="-0.20 0.0 0.31 3.14159 0.7 0.0 base_link usb_cam_optical_frame"` (x y z yaw pitch roll). Это камера спереди по центру, ~0.31 м над `base_link` , смотрит назад-вниз. **Подгоните эти 6 чисел под реальное крепление вашей камеры**, иначе позиция кубика в `odom` будет неточной. Также задайте калибровку камеры через `camera_info_url` (закомментированная строка в launch).

В.5 Свой размер комнаты и ячейки на реальном роботе

Если на реальной сцене нужны точки/препятствия из `room_generator` , включите его и задайте размеры под РЕАЛЬНОЕ пространство:

- **[НОУТБУК]** `bash roslaunch youbot_perception pick_place_real.launch \ video_device:=/dev/video0 \ use_room_generator:=true \ room_width:=3.0 room_height:=2.5 cell_size:=0.5 \ cell_A:=0,0 cell_B:=2,4 cell_C:=5,4`
- `room_width / room_height` — реальные размеры комнаты в метрах (измерьте рулеткой!);
- `cell_size` — желаемый размер ячейки;
- `cell_A/B/C` — ячейки парковки/кубика/склада в формате `col,row` .

Робот при этом ДОЛЖЕН физически стоять в точке, соответствующей `cell_A` (см. В.0). origin комнаты по умолчанию (0,0) = стартовая поза робота.

Если кубик и склад вы расставляете руками и `room_generator` не нужен — не включайте его (`use_room_generator:=false`, по умолчанию). Тогда точки A/B/C миссия возьмёт из `mission_params.yaml` либо из своих параметров. В этом случае убедитесь, что координаты в конфиге заданы в метрах в кадре `odom` относительно стартовой позы робота.

В.6 Порядок включения (резюме)

1. **[YOUBOT]** включить базу/манипулятор, проверить EtherCAT-кабель;
2. **[НОУТБУК]** `ping` робота;
3. поставить робот в точку А правильно ориентированным (B.0);
4. **[YOUBOT]** `roslaunch youbot_driver_ros_interface youbot_driver.launch`;
5. **[НОУТБУК]** `roslaunch youbot_perception pick_place_real.launch ...`;
6. наблюдать `/cube/debug image`, `/cube/color`, `/mission planned path`.

В.7 Реальная рабочая последовательность (ssh-раскладка, проверено на стенде)

Это конкретный порядок запуска, отражающий фактическую установку, где **ROS-мастер и драйвер крутятся на самом роботе**, а к роботу заходят по `ssh alisa@10.0.0.1`. Камера тоже поднимается на роботе (через `roslaunch usb_cam`), а программа миссии и просмотр изображения — с ноутбука.

Про source ... setup.bash : если у вас он прописан в `~/bashrc`, то для ИНТЕРАКТИВНЫХ сессий он подхватится сам. Но `ssh user@host "команда"` запускает НЕинтерактивную оболочку, и `~/bashrc` может НЕ прочитаться. Поэтому ниже `source` указан явно в каждом терминале — это безопаснее и работает всегда. Подставьте свои реальные пути к воркспейсам (`~/rosws_kvant`, `~/catkin_ws` и т.п.).

Про ROS-мастер: мастер здесь — на роботе (<http://10.0.0.1:11311>). Все ноды (и на роботе, и на ноутбуке) должны указывать на него. На ноутбуке НЕ нужно поднимать второй `roscore` — иначе ноды ноутбука уйдут на локальный мастер и не увидят робота. Вместо отдельного `roscore` на ноутбуке просто выставьте `ROS_MASTER_URI` на робота (см. ниже). Если вы всё же привыкли запускать `roscore` на ноутбуке — тогда мастером становится ноутбук, и уже робот должен смотреть на него; держите ОДИН мастер на всех.

Про source ... setup.bash : если у вас он прописан в `~/bashrc`, то для ИНТЕРАКТИВНЫХ сессий он подхватится сам. Но `ssh user@host "команда"` запускает НЕинтерактивную оболочку, и `~/bashrc` может НЕ прочитаться. Поэтому ниже `source` указан явно в каждом терминале — это безопаснее и работает всегда. Подставьте свои реальные пути к воркспейсам (`~/rosws_kvant`, `~/catkin_ws` и т.п.).

Про ROS-мастер: мастер здесь — на роботе (<http://10.0.0.1:11311>). Все ноды (и на роботе, и на ноутбуке) должны указывать на него. На ноутбуке НЕ нужно поднимать второй `roscore` — иначе ноды ноутбука уйдут на локальный мастер и не увидят робота. Вместо отдельного `roscore` на ноутбуке просто выставьте `ROS_MASTER_URI` на робота (см. ниже). Если вы всё же привыкли запускать `roscore` на ноутбуке — тогда мастером становится ноутбук, и уже робот должен смотреть на него; держите ОДИН мастер на всех.

Терминал 1 — [YOUBOT] ROS-мастер на работе

```
ssh alisa@10.0.0.1
source ~/rosws_kvant/devel/setup.bash      # ваш путь к воркспейсу на работе
export ROS_MASTER_URI=http://10.0.0.1:11311
export ROS_IP=10.0.0.1
roscore
```

Терминал 2 — [YOUBOT] драйвер робота

```
ssh alisa@10.0.0.1
source ~/rosws_kvant/devel/setup.bash
export ROS_MASTER_URI=http://10.0.0.1:11311
export ROS_IP=10.0.0.1
roslaunch youbot_driver_ros_interface youbot_driver.launch
```

(если тут сыпется `EtherCAT No base initialized / No socket connection` — см. раздел D.1; чаще всего это имя интерфейса в `youbot-ethercat.cfg` и/или `setcap` на бинарь драйвера)

Терминал 3 — [YOUBOT] камера (с пробросом X для GUI)

```
ssh -X alisa@10.0.0.1
source ~/rosws_kvant/devel/setup.bash
export ROS_MASTER_URI=http://10.0.0.1:11311
export ROS_IP=10.0.0.1
roslaunch usb_cam usb_cam_node \
  _video_device:=/dev/video0 \
  _pixel_format:=yuyv \
  _camera_frame_id:=usb_cam_optical_frame \
  _image_width:=1280 _image_height:=720
```

Терминал 4 — [НОУТБУК] просмотр изображения камеры

[illegible]

Если прямая трансляция картинки не работает (частый случай при слабой сети или X-пробросе) — сохраняйте кадр в файл и открывайте его:

```
# [YOUBOT] (через ssh -X), пишем кадр в test.jpg
ssh -X alisa@10.0.0.1
source ~/rosws_kvant/devel/setup.bash
export ROS_MASTER_URI=http://10.0.0.1:11311
export ROS_IP=10.0.0.1
roslaunch image_view image_view image:=/usb_cam/image_raw _filename:=test.jpg
```

Терминал 5 — [НОУТБУК] запуск программы миссии с медленными скоростями

```
source ~/catkin_ws/devel/setup.bash
export ROS_MASTER_URI=http://10.0.0.1:11311
export ROS_IP=<IP_ноутбука_в_сети_робота>
cd ~/catkin_ws/src/RMPC-project-KUKA-youBot # ваш путь к проекту
roslaunch youbot_perception pick_place_mission.py \
  _v_max:=0.10 _w_max:=0.4 _v_creep:=0.03
```

Параметры `_v_max`, `_w_max`, `_v_creep` здесь заданы как ПРИВАТНЫЕ параметры ноды (`_имя:=значение`).

Благодаря хелперу `mission_config` приватный параметр имеет приоритет над единым конфигом, так что эти значения переопределяют `mission_params.yaml` именно для данного запуска (см. также раздел C.2).

Запуск через `roslaunch` НЕ поднимает `cube_detector` и `room_generator` — их при необходимости запусайте отдельными терминалами (как в B.4/B.5) или используйте `pick_place_real.launch`, который поднимает камеру, детектор и миссию вместе.

Обновление кода проекта из git (когда нужно подтянуть ветку):

```
# на той машине, где лежит проект (обычно [НОУТБУК])
cd ~/catkin_ws/src/RMPC-project-KUKA-youBot
git pull origin branch_1
# после изменений в C++/драйвере — пересобрать и заново выдать setcap (см. D.1)
```

B.8 Визуализация и сравнение с симуляцией (на реальном роботе)

- **[НОУТБУК]** Откройте RViz с готовой сценой к уже запущенной миссии: `bash source ~/catkin_ws/devel/setup.bash export ROS_MASTER_URI=http://10.0.0.1:11311 export ROS_IP=<IP_ноутбука_в_сети_робота> roslaunch youbot_controller rviz.launch` Увидите фактический путь робота (красный, `/mission_actual_path`, строится из `/odom`), запланированный путь (зелёный) и, если запущен `room_generator`, комнату с препятствиями и точками A/B/C.
- Чтобы СРАВНИТЬ реальность с симуляцией: запишите видео RViz/камеры на реальном прогоне и видео из Gazebo (`pick_place_sim.launch rviz:=true`) с теми же параметрами комнаты/ячеек. Красный фактический путь в обоих случаях берётся из `/odom`, поэтому траектории напрямую сопоставимы. См. подробности в A.8.

Примечание: на реальном роботе модель робота в RViz (RobotModel) появится только если на параметр-сервер загружен `robot_description` — его поднимает `youbot_driver.launch`. Если модели нет, остальные слои (пути, маркеры, картинка камеры) всё равно работают.

C. Короткие команды для теста

Все команды ниже — **[НОУТБУК]** (если не указано иное). Перед ними в каждом терминале выставьте сетевые переменные ROS (B.2) и `source devel/setup.bash`.

C.1 Раскрытие схвата на максимум

Схват слушает топик `/gripper_controller/command` (`trajectory_msgs/JointTrajectory`), суставы `gripper_finger_joint_l` и `gripper_finger_joint_r`, позиция В МЕТРАХ НА ПАЛЕЦ.

Реальный робот — максимум ~7 см (0.035 м на палец):

```
rostopic pub -1 /gripper_controller/command trajectory_msgs/JointTrajectory \
  "joint_names: ['gripper_finger_joint_l', 'gripper_finger_joint_r']
  points:
  - positions: [0.035, 0.035]
    time_from_start: {secs: 2, nsecs: 0}"
```

Симуляция — максимум по URDF (0.0115 м на палец):

```
rostopic pub -1 /gripper_controller/command trajectory_msgs/JointTrajectory \
"joint_names: ['gripper_finger_joint_l', 'gripper_finger_joint_r']
points:
- positions: [0.0115, 0.0115]
  time_from_start: {secs: 2, nsecs: 0}"
```

Закрывать хват полностью (оба случая): замените позиции на `[0.0, 0.0]`.

С.2 Запуск с медленными скоростями

Скорости задаются в `mission_params.yaml` (группа `mission`): `v_max`, `w_max`, `v_creep`. Для безопасного теста уменьшите их прямо в команде запуска — параметры миссии можно переопределить приватно через namespace ноды:

Симуляция, медленно:

```
roslaunch youbot_perception pick_place_sim.launch \
  cube_color:=red &
sleep 12
rosparam set /pick_place_mission/v_max 0.08
rosparam set /pick_place_mission/w_max 0.4
rosparam set /pick_place_mission/v_creep 0.03
```

Параметры миссии читаются при старте ноды, поэтому надёжнее задать их ДО запуска. Самый чистый способ — временно поправить `mission_params.yaml` (`v_max: 0.08`, `w_max: 0.4`, `v_creep: 0.03`) и запустить обычным образом.

Ручное управление базой на медленной скорости (телеоп) для проверки движения:

- симуляция или реальный робот, база слушает `/cmd_vel`: `bash rosrunc youbot_driver_ros_interface youbot_keyboard_teleop.py` Управление клавишами i/j/l/k (вперёд/поворот вправо/поворот влево/стоп). **ВАЖНО:** сначала снизьте скорость клавишей `z`.
- либо одиночная медленная команда «вперёд 0.05 м/с» на 1 с: `bash rostopic pub -1 /cmd_vel geometry_msgs/Twist \
"linear: {x: 0.05, y: 0.0, z: 0.0} angular: {x: 0.0, y: 0.0, z: 0.0}"`

С.3 Проверка камеры и детекции

1. Идёт ли поток с камеры: `bash rostopic hz /usb_cam/image_raw` # должна быть ненулевая частота `roslaunch rqt_image_view rqt_image_view /usb_cam/image_raw`
2. Работает ли детектор (видит ли кубик и его цвет): `bash roslaunch rqt_image_view rqt_image_view /cube/debug_image` # обводка кубика `rostopic echo /cube/color` # red/green/blue `rostopic echo /cube/pose` # позиция в odom
3. Есть ли калибровка камеры (нужна для точной позиции): `bash rostopic echo -n1 /usb_cam/camera_info` # K не должна быть нулевой
4. Проверка детекции БЕЗ запуска всей миссии (только камера + детектор): - **[НОУТБУК]** `bash roslaunch youbot_perception cube_detector.py \
_image_topic:=/usb_cam/image_raw \
_info_topic:=/usb_cam/camera_info \
_target_frame:=odom` - поднесите кубик в кадр и смотрите `/cube/color` и `/cube/debug_image`. - если цвет «прыгает» или не виден — подгоните HSV-пороги в `cube_detector.py` (`COLOR_RANGES`) под ваше освещение.

D. Трешшутинг частых проблем

D.1 EtherCAT (самая частая проблема реального робота)

Драйвер бросает одно из трёх исключений. Полный разбор — `docs/ETHERCAT_TROUBLESHOOTING.md`. Кратко:

No socket connection on eth0 / Execute as root — драйвер не открыл сокет. Две причины: 1. **Неверное имя интерфейса**. На современной Ubuntu интерфейс называется не `eth0`, а, например, `enp2s0` / `eno1`. - **[YOUBOT]** узнать имя: `ip link` - **[YOUBOT]** прописать в `youbot_driver/config/youbot-ethercat.cfg`: `EthernetDevice = enp2s0` 2. **Нет прав на RAW-сокет**. Не запускайте из-под `sudo` (ломает окружение). Выдайте capability бинарию (повторять ПОСЛЕ КАЖДОЙ пересборки): - **[YOUBOT]** `bash sudo setcap cap_net_raw,cap_net_admin+eip \ $(catkin_find youbot_driver_ros_interface youbot_driver_ros_interface | head -1)` `getcap <путь-к-бинарию>` # должно показать `cap_net_raw`

No EtherCAT slaves found! — сокет открылся, но моторов на линии нет: - **[YOUBOT]** база не включена/разряжена → включить, зарядить свинцовый аккумулятор; - проверить EtherCAT-кабель и линк: `ip link` должен показывать интерфейс `UP`, `LOWER_UP`; - на этот интерфейс не должен идти посторонний IP-трафик (выделенный порт под робота).

Not all EtherCAT slaves reached operational state — slave'ы найдены, но не вышли в OP: - сначала включить/разбудить базу, ПОТОМ запускать драйвер (гонка при старте); - увеличить таймауты в `youbot-ethercat.cfg`

(`EtherCATTimeout_[usec]` , `MailboxTimeout_[usec]`); - лог `no distributed clock is available` сам по себе не фатален.

D.2 Инициализация робота / драйвер не стартует

- **Робот стоит не там / координаты «уехали».** `odom` создаётся при старте драйвера в текущей позе робота. Если робот поставлен неправильно — пересоберите сцену: остановите драйвер, поставьте робот в точку А (В.0), запустите драйвер заново.
- **Рука не инициализируется.** В `youbot_driver.launch` параметр `youBotHasArms` по умолчанию `false`. Если нужен манипулятор от драйвера — [YOUBOT] установите `true` и задайте имя руки (`youBotArmName1`), под которое есть конфиг в `youbot_driver/config`.
- **Контроллеры не поднялись (симуляция).** Проверьте, что Gazebo не на паузе и `controller_manager` запущен: `bash rosservice call /controller_manager/list_controllers rostopic echo /joint_states` # должны идти позиции суставов

D.3 Робот не едет / едет не туда

- **Нет `/odom`.** Миссия ждёт одометрию и не стартует без неё: `bash rostopic hz /odom` Нет данных → в симуляции не поднялся `planar_move` (проверьте URDF-плагин); на реальном — не поднялся драйвер базы.
- **База игнорирует `/cmd_vel`.** Проверьте, что публикация доходит: `bash rostopic echo /cmd_vel`
- **RRT всегда «едет напрямую».** Это нормально, если точка реально недостижима (например, у самой стены). Если объезд нужен — проверьте, что препятствия публикуются: `rostopic echo /room/obstacles` (должен быть непустой массив) и что запущен `room_generator`.

D.4 Детекция цвета не работает

- **`/cube/color` пустой.** Камера не видит кубик или HSV-пороги не подходят:
- проверьте поток (`rostopic hz /usb_cam/image_raw`);
- смотрите `/cube/debug_image` — обводит ли детектор кубик;
- подгоните `COLOR_RANGES` в `cube_detector.py` под освещение.
- **Кубик в «слепой зоне».** Камера наклонена вниз и видит пол в полосе ~0.47– 1.39 м перед базой; ближе кубик не виден. Миссия сама встаёт на дистанцию осмотра (`view_dist` , по умолчанию 0.73 м).
- **Миссия не стопорится при провале детекции.** Это сделано намеренно: после `color_attempts` попыток (по умолчанию 3) миссия берёт `fallback_color` (по умолчанию red) и продолжает. Меняется в `mission_params.yaml`.

D.5 Захват не удаётся

- **Схват не дотягивается до кубика.** На реальном роботе подгоните `arm_reach` и `view_dist` в конфиге под вашу руку/камеру.
- **Кубик не «прилипает» в Gazebo.** Нужен плагин `grasp-fix` (`libgazebo_grasp_fix.so`); без него симулированный схват кубик не удержит.
- **Схват слишком слабо/сильно жмёт (реальный робот).** Кубик ~5 см; схват обхватывает его за ребро. При необходимости отрегулируйте позиции открытия/ закрытия (раздел С.1) и выберите усилие.

D.6 Сетевые проблемы ROS (ноутбук ↔ youBot)

- **Ноды на ноутбуке не видят топики робота.** Проверьте на ОБЕИХ машинах: `bash echo $ROS_MASTER_URI # одинаковый, указывает на мастер echo $ROS_IP # IP именно этой машины в общей сети rostopic list # должны быть видны топики робота`
- **TF-ошибки `usb_cam_optical_frame → odom`.** Не задан или неверен статический TF камеры (В.4). Проверьте дерево TF: `bash roslaunch tf2_tools view_frames.py && evince frames.pdf`

Приложение: шпаргалка значений по умолчанию

Параметр	Значение	Где менять
Размер комнаты	5.0 × 5.0 м	<code>room/width</code> , <code>room/height</code> или <code>arg launch</code>
Размер ячейки	1.0 м	<code>room/cell_size</code> или <code>arg</code>
Ячейка А (парковка)	авто (угол 0,0)	<code>points/cell_A</code> или <code>arg</code>
Ячейка В (кубик)	0,4 (левый верхний угол)	<code>points/cell_B</code> или <code>arg</code>
Ячейка С (склад)	авто (дальний угол)	<code>points/cell_C</code> или <code>arg</code>
Радиус препятствия	0.1 м (фикс.)	<code>obstacles/radius_min / max</code>
Разброс препятствий	1.0	<code>obstacles/spread</code>
Частота препятствий	0..2 на ячейку	<code>obstacles/max_per_cell</code>
seed	42	<code>room/seed</code> или <code>arg</code>
Радиус робота	0.30 м	<code>robot/robot_radius</code>
<code>v_max / w_max / v_creep</code>	0.22 / 1.0 / 0.05	<code>mission/*</code>
Попыток детекции	3, затем fallback red	<code>mission/color_attempts</code> , <code>cube/fallback_color</code>
Схват макс (sim / real)	0.0115 / ~0.035 м на палец	команда в С.1

Топики визуализации (для RViz)

Топик	Что показывает	Кто публикует
<code>/room/markers</code>	комната, стены, сетка, препятствия, точки А/В/С	<code>room_generator</code>
<code>/field/markers</code>	угловые маркеры рабочей зоны	<code>cube_detector</code>
<code>/mission_planned_path</code>	запланированный путь RRT (зелёный)	<code>pick_place_mission</code>
<code>/mission_actual_path</code>	фактически пройденный путь (красный, из <code>/odom</code>)	<code>pick_place_mission</code>
<code>/cube/debug_image</code>	кадр камеры с обводкой кубика	<code>cube_detector</code>

Готовый RViz-конфиг: `youbot_controller/config/mission_view.rviz` (запуск: `roslaunch youbot_controller rviz.launch` или `pick_place_sim.launch rviz:=true`).